

Practical No : 2

Aim : To Implement Logistic Regression

```
[10] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
```

```
✓ [2] # Sample dataset (can replace with CSV)
js data = {
    'Age': [22, 25, 47, 52, 46, 56, 55, 60, 62, 61],
    'Salary': [15000, 29000, 48000, 60000, 52000, 61000, 58000, 63000, 64000, 65000],
    'Purchased': [0, 0, 1, 1, 1, 1, 1, 1, 1, 1]
}
```

```
[3] # Convert to DataFrame
df = pd.DataFrame(data)

# Independent and Dependent variables
X = df[['Age', 'Salary']] # Features
y = df['Purchased']       # Target
```

```
✓ [4] # Split the data into training and testing sets
js X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create logistic regression model
model = LogisticRegression()

# Train the model
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)
```

```
✓ # Evaluation
js print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nAccuracy Score:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

⇒ Confusion Matrix:

```
[[1 0]
 [0 1]]
```

Accuracy Score: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

```
[14] new_person = pd.DataFrame([[30, 40000]], columns=['Age', 'Salary'])  
prediction = model.predict(new_person)  
  
print("\nPrediction for Age=30 & Salary=40000:", "Purchased" if prediction[0] == 1 else "Not Purchased")
```



Prediction for Age=30 & Salary=40000: Purchased